LAPORAN JOBSHEET
“8-Client Side Rendering & Data Fetching”
MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

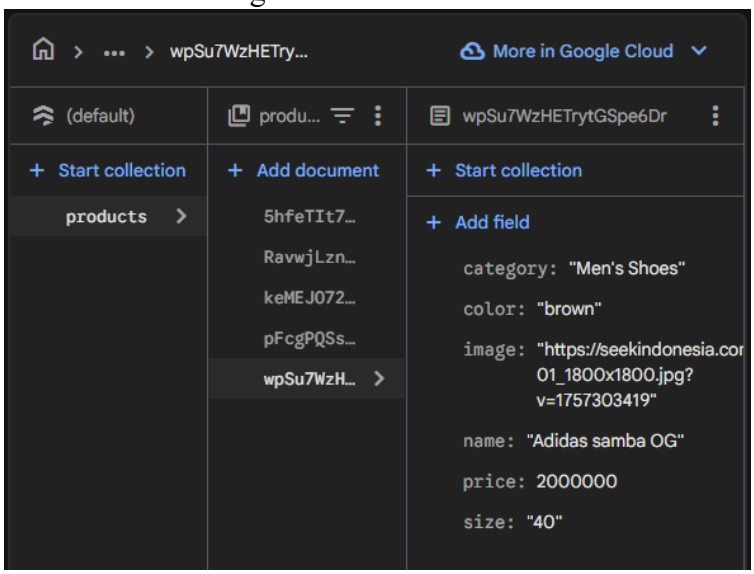
PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

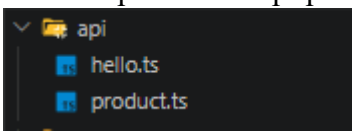
2026

Menambahkan image address ke dalam field baru



Bagian 1 – Setup Data Produk

1. Siapkan project Next.js.
2. Buat endpoint API /api/products.



3. Pastikan data memiliki:
   - id
   - name
   - category
   - price
   - image

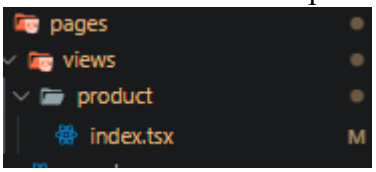
```
type ProductType = {
  id: string;
  name: string;
  price: number;
  size: string;
  color: string;
  category: string;
};
```

4. Jalankan browser http://localhost:3000/api/product


```
{
    "status": true,
    "statusCode": 200,
    "data": [
        {
            "id": "5hfeTIt7HIyrCmmnokNa",
            "category": "jacket",
            "name": "Nike Crewneck ",
            "size": "XL",
            "price": 150000,
            "color": "black"
        },
        {
            "id": "RavwjLznBePFObOkBhLR",
            "price": 4000000,
            "color": "white",
            "category": "shoes",
            "size": "39-45",
            "name": "Adidas Adizero Adios Pro 4"
        },
        {
            "id": "keMEJO724VcBe9w5Fx3m",
            "color": "brown",
            "price": 6399000,
            "category": "jacket",
            "name": "Carhartt Detroit Jacket",
            "image": "https://www.bfgcdn.com/1500_1500_90/003-3004/carhartt-relaxed-fit-duck-detroit-jacket-casual-jacket.jpg",
            "size": "M, L, XL"
        },
        {
            "id": "pFcgPQSsXREvHumHlXKh",
            "category": "shoes",
            "color": "red",
            "price": 1299000,
            "size": "40 - 45",
            "name": "Vans Auth Red Chili"
        },
        {
            "id": "wpSu7WzHETrytGSpe6Dr",
            "image": "https://seekindonesia.com/cdn/shop/files/ID1447-01_1800x1800.jpg?v=1757303419",
            "size": "40",
            "name": "Adidas samba OG",
            "category": "Men's Shoes",
            "color": "brown",
            "price": 2000000
        }
    ]
}
```


Bagian 2 – Implementasi CSR dengan useEffect

1. Membuat file index.tsx pada folder views/products

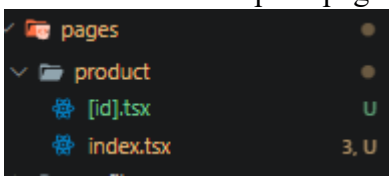


2. Modifikasi index.tsx

3. P2 > Praktikum > next-routing > src > pages > views > product > index.tsx > ShowProducts > products.map() callback

```
import { useState, useEffect } from "react";

type ProductType = {
  id: string;
  name: string;
  price: number;
  size: string;
  color: string;
  category: string;
  image: string;
};
const useCategory = () => {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(false);

  const fetchProducts = async () => {
    setLoading(true);
    try {
      const response = await fetch("/api/product");
      const data = await response.json();
      setProducts(data.data);
    } catch (error) {
      console.error("Error fetching products:", error);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchProducts();
  }, []);

  return {
    products,
    loading,
    refreshProducts: fetchProducts,
  };
};
```

Buka file index.tsx pada pages/product/

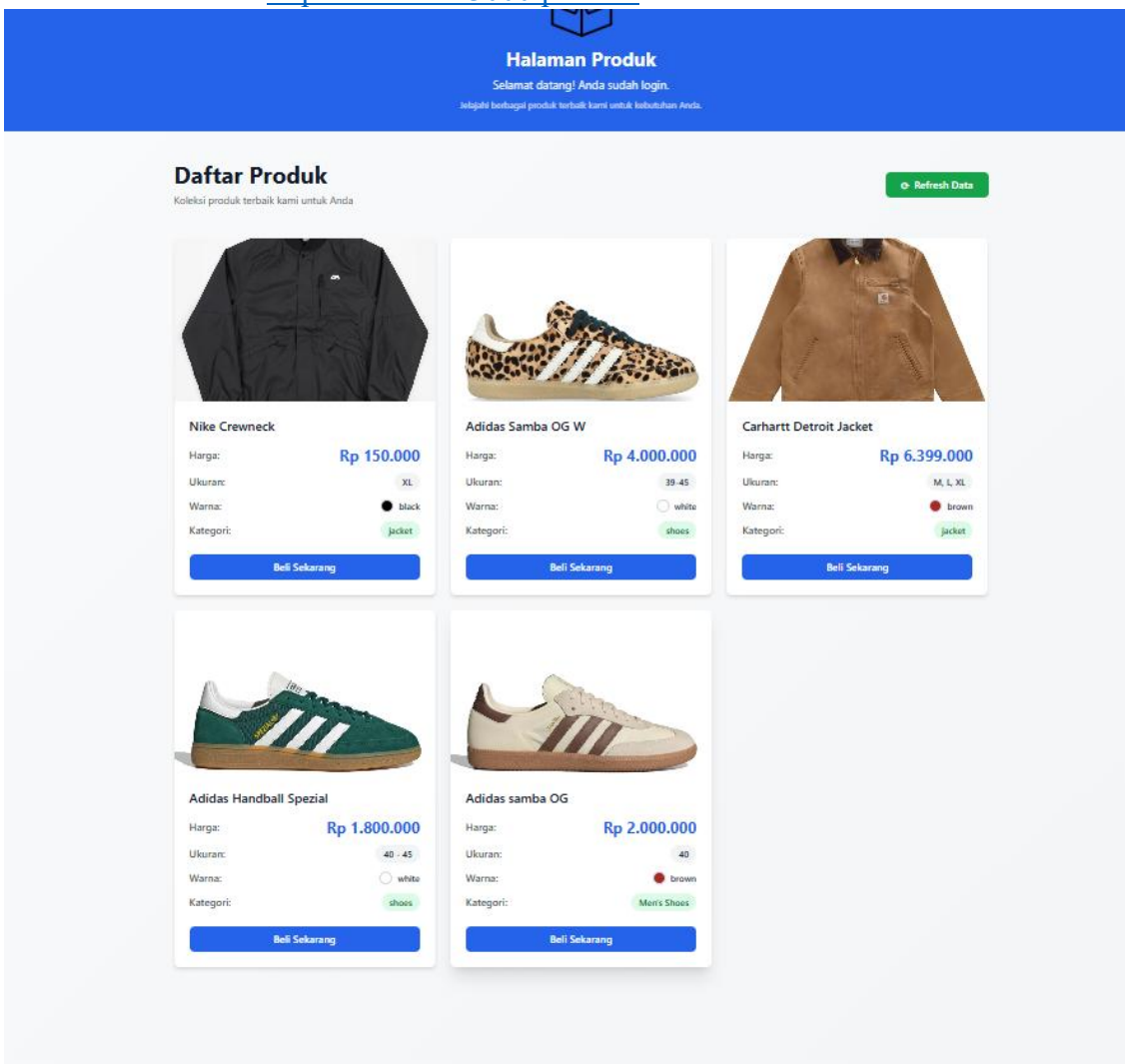


4. Modifikasi index.tsx pada pages/product/

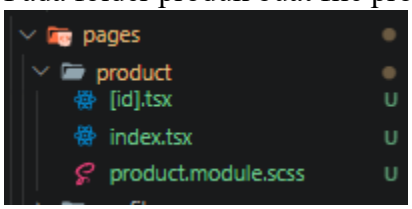
P2 > Praktikum > next-routing > src > pages > product > index.tsx > ProductPage

```
import Head from "next/head";
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import HeroSection from "../views/product/components/HeroSection";
import MainSection from "../views/product/components/MainSection";
import ShowProducts from "../views/product";

const ProductPage = () => {
  const { push } = useRouter();
  const [isLoading, setIsLoading] = useState(true);

  // useEffect(() => {
  //   const loginStatus = localStorage.getItem("isLogin");
  //   if (loginStatus !== "true") {
  //     push("/auth/login");
  //   } else {
  //     // eslint-disable-next-line
  //     setIsLoading(false);
  //   }
  // }, [push]);

  const handleLogout = () => {
    localStorage.removeItem("isLogin");
    push("/auth/login");
  };

  // if (isLoading) {
  //   return <p className="text-center text-gray-500 mt-10">Loading...</p>;
  // }

  return (
    <>
      <Head>
        <title>Halaman Produk | My App</title>
      </Head>
      <div className="min-h-screen bg-gray-100">
        <HeroSection />
        <ShowProducts />
        <MainSection onLogout={handleLogout} />
      </div>
    </>
  );
};

export default ProductPage;
```

5. Jalankan di browser http://localhost:3000/product



6. Pada folder produk buat file produk.modules.scss



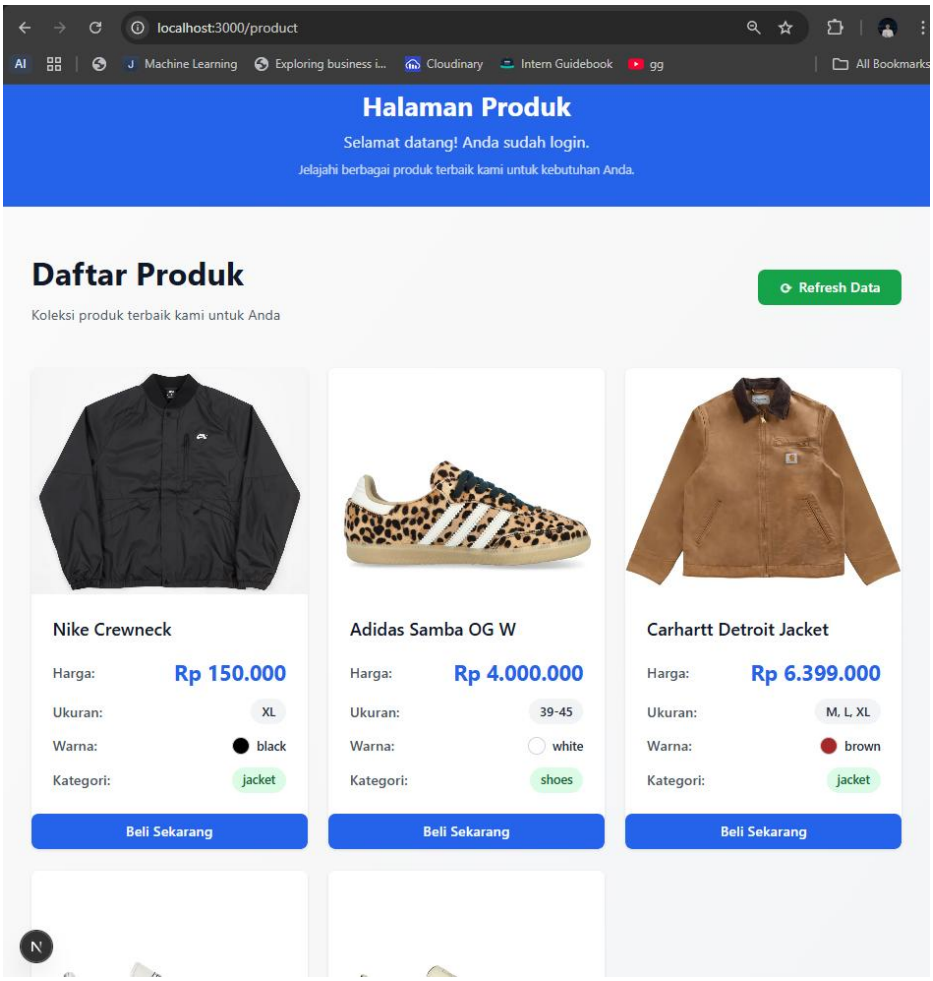
7. Modifikasi produk.modules.scss

```
P2 > Praktikum > next-routing > src > pages > product > product.module.scss > ...

// Product Wrapper
.product_wrapper {
  display: flex;
}

// Product Card
.product_card {
  background-color: white;
  border-radius: 0.5rem;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
  transition: box-shadow 0.3s ease;
  overflow: hidden;
  width: 100%;
  display: flex;
  flex-direction: column;

  &:hover {
    box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
  }
}

// Product Image
.product_image {
  width: 100%;
  height: 16rem;
  object-fit: cover;
  display: block;
}

// Product Content
.product_content {
  padding: 1.5rem;
  flex: 1;
  display: flex;
  flex-direction: column;
}

// Product Title Section
.product_title_subtitle {
  margin-bottom: 1rem;
}

.product_title {
  font-size: 1.25rem;
  font-weight: 600;
  color: #111827;
```

8. Modifikasi Pada file index.tsx pada folder pages/views/product

```
<div className={styles.grid}>
  {products.map((product: ProductType) => (
    <div key={product.id} className={styles.product_wrapper}>
      <div className={styles.product_card}>
        <img
          src={product.image}
          alt={product.name}
          className={styles.product_image}
        />
        <div className={styles.product_content}>
          <div className={styles.product_title_subtitle}>
            <h2 className={styles.product_title}>{product.name}</h2>
          </div>
          <div className={styles.product_content_item}>
            <div className={styles.product_info}>
              <p className={styles.product_label}>Harga:</p>
              <span className={styles.product_price}>
                Rp {product.price.toLocaleString("id-ID")}
              </span>
            </div>
          </div>
          <div className={styles.product_content_item}>
            <div className={styles.product_info}>
              <p className={styles.product_label}>Ukuran:</p>
              <span className={styles.product_size}>
                {product.size}
              </span>
            </div>
          </div>
          <div className={styles.product_content_item}>
            <div className={styles.product_info}>
              <p className={styles.product_label}>Warna:</p>
              <div className={styles.product_color_container}>
                <div
                  className={styles.product_color_circle}
                  style={{
                    backgroundColor: product.color.toLowerCase(),
                  }}
                ></div>
                <span className={styles.product_color_label}>
                  {product.color}
                </span>
              </div>
            </div>
          </div>
          <div className={styles.product_content_item}>
```

9. Jalankan Browser



Bagian 3 – Implementasi Skeleton Loading

1. Modfikasi file index.tsx pada folder views/product/index.tsx

```
const ProductSkeletonLoader = () => {
  return (
    <div className={styles.product_wrapper}>
      <div className={styles.product_card}>
        <div className={styles.skeleton_image}></div>
        <div className={styles.product_content}>
          <div className={styles.product_title_subtitle}>
            <div className={styles.skeleton_title}></div>
          </div>
          <div className={styles.product_content_item}>
            <div className={styles.product_info}>
              <div className={styles.skeleton_label}></div>
              <div className={styles.skeleton_value}></div>
            </div>
          </div>
          <div className={styles.product_content_item}>
            <div className={styles.product_info}>
              <div className={styles.skeleton_label}></div>
              <div className={styles.skeleton_badge}></div>
            </div>
          </div>
          <div className={styles.product_content_item}>
            <div className={styles.product_info}>
              <div className={styles.skeleton_label}></div>
              <div className={styles.skeleton_color_container}>
                <div className={styles.skeleton_color_circle}></div>
                <div className={styles.skeleton_color_label}></div>
              </div>
            </div>
          </div>
          <div className={styles.product_content_item}>
            <div className={styles.product_info}>
              <div className={styles.skeleton_label}></div>
              <div className={styles.skeleton_badge}></div>
            </div>
          </div>
        </div>
        <div className={styles.skeleton_button}></div>
      </div>
    </div>
  );
};
```

Karena saya ingin mempertahankan design yg sebelumnya sudah saya buat, kemudian saya harus mengimplementasikan skeleton loading maka, saya membuatnya menjadi file berbeda (ProdutSkeletonLoader.tsx).

```
{loading ? (
  <div className={styles.grid}>
    {skeletonArray.map((_, index) => (
      <ProductSkeletonLoader key={`skeleton-${index}`} />
    ))}
  </div>
```

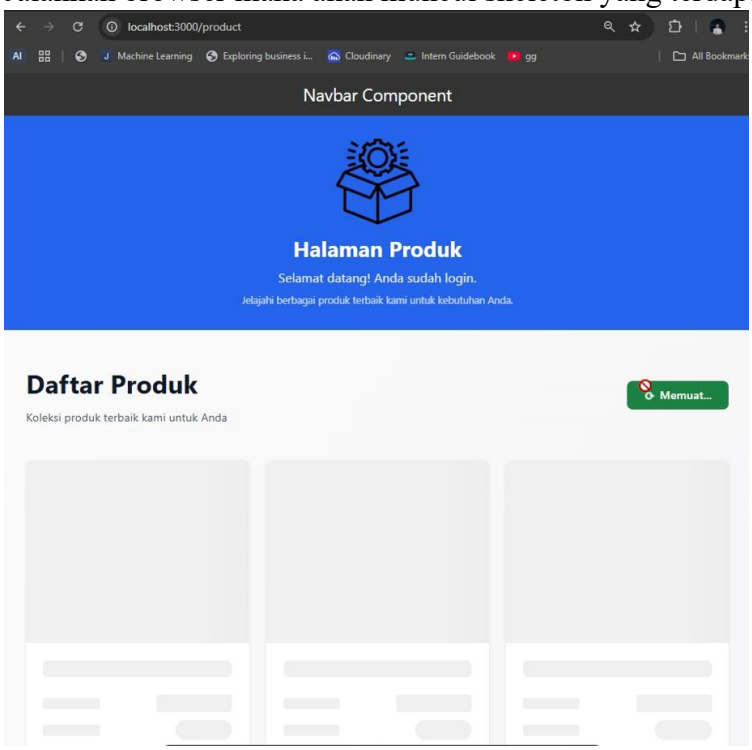
Jadi ketika loading maka ProductSkeletonLoadernya dipanggil seperti ini (kode ini ada di file index.tsx di folder views/product

2. Modifikasi file product.module.scss

```scss
.skeleton_image {
  width: 100%;
  height: 16rem;
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 1000px 100%;
  animation: shimmer 2s infinite;
  display: block;
}

.skeleton_title {
  height: 1.5rem;
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 1000px 100%;
  animation: shimmer 2s infinite;
  border-radius: 0.5rem;
  margin-bottom: 0.5rem;
}

.skeleton_label {
  width: 30%;
  height: 1rem;
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 1000px 100%;
  animation: shimmer 2s infinite;
  border-radius: 0.25rem;
}

.skeleton_value {
  width: 40%;
  height: 1.5rem;
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
  background-size: 1000px 100%;
  animation: shimmer 2s infinite;
  border-radius: 0.25rem;
}

.skeleton_badge {
  width: 30%;
  height: 1.75rem;
  background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%);
```

MS 11 OUTPUT DEBUG CONSOLE TERMINAL PORTS CONSOLE NINJA

3. Jalankan browser maka akan muncul skeleton yang terdapat animasi berkedip



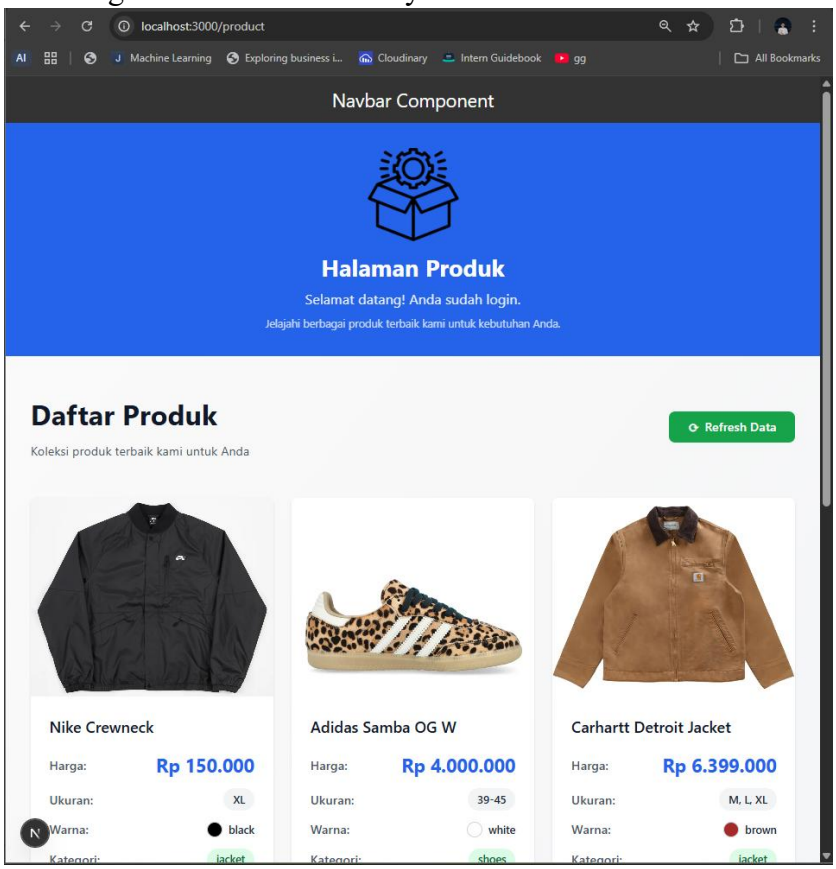
4. Modifikasi pada index.tsx pada folder views/product/index.tsx dan ProductSkeletonLoader.tsx di saya

```
product.module.scss U   index.tsx 2, M ×   ProductSkeletonLoader.tsx U
P2 > Praktikum > next-routing > src > pages > views > product > index.tsx > ShowProducts >
45  const ShowProducts = () => {
75            </button>
76          </div>
77
78          {loading ? (
79            <div className={styles.grid}>
80              {skeletonArray.map((_, index) => (
81                <ProductSkeletonLoader key={`skeleton-${index}`} />
82              ))}
83            </div>
```

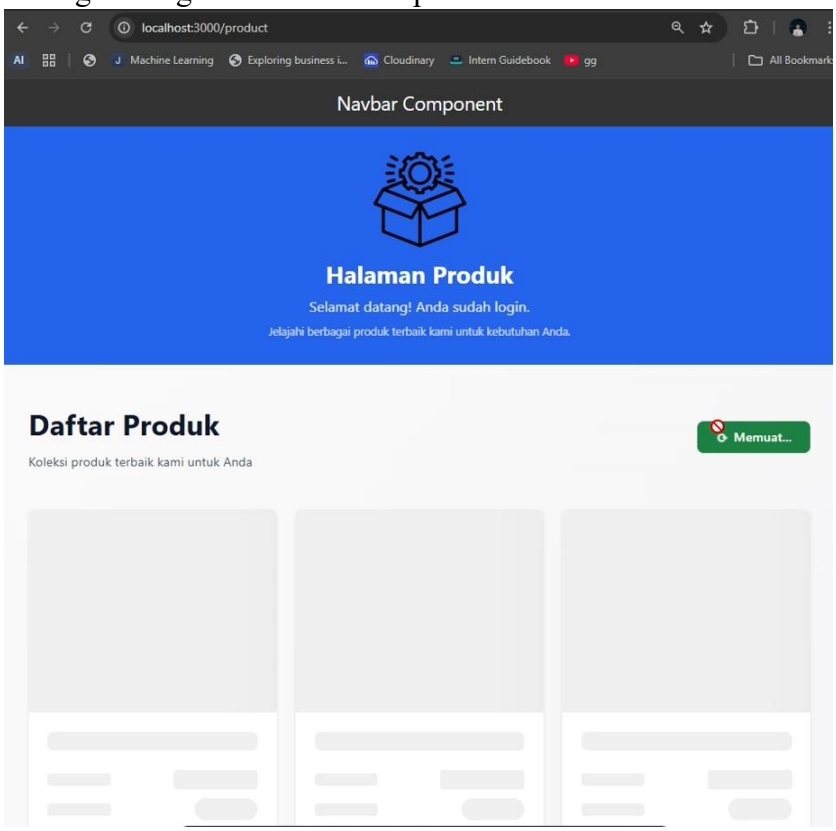
ProductSkeletonLoader.tsx

```
product.module.scss U   index.tsx 2, M   ProductSkeletonLoader.tsx U ×
P2 > Praktikum > next-routing > src > pages > views > product > components > ProductSkeletonLoader.ts
11  const ProductSkeletonLoader = () => {
12    return (
13      <div className={styles.product_wrapper}>
14        <div className={styles.product_card}>
15          <div className={styles.skeleton_image}></div>
16          <div className={styles.product_content}>
17            <div className={styles.product_title_subtitle}>
18              <div className={styles.skeleton_title}></div>
19            </div>
20            <div className={styles.product_content_item}>
21              <div className={styles.product_info}>
22                <div className={styles.skeleton_label}></div>
23                <div className={styles.skeleton_value}></div>
24              </div>
25            </div>
26            <div className={styles.product_content_item}>
27              <div className={styles.product_info}>
28                <div className={styles.skeleton_label}></div>
29                <div className={styles.skeleton_badge}></div>
30              </div>
31            </div>
32            <div className={styles.product_content_item}>
33              <div className={styles.product_info}>
34                <div className={styles.skeleton_label}></div>
35                <div className={styles.skeleton_color_container}>
36                  <div className={styles.skeleton_color_circle}></div>
37                  <div className={styles.skeleton_color_label}></div>
38                </div>
39              </div>
40            </div>
41            <div className={styles.product_content_item}>
42              <div className={styles.product_info}>
43                <div className={styles.skeleton_label}></div>
44                <div className={styles.skeleton_badge}></div>
45              </div>
46            </div>
47          </div>
48          <div className={styles.skeleton_button}></div>
49        </div>
50      </div>
51    );
52  };
```

5. Jalankan browser. Jika dijalankan akan muncul skeletonnya terlebih dahulu setelah itu muncul gambar dan informasinya



Ditengah Tengah reload akan seperti ini:



Bagian 5 – Implementasi SWR

1. Install SWR

```
PS D:\KULIAH\SEMESTER-6\Pemrograman-Berbasis-Framework\P2\Praktikum\next-routing> npm install swr

added 3 packages, and audited 473 packages in 6s

154 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

2. Buka dan modifkasi file index.tsx pada folder pages/product/

```
P2 > Praktikum > next-routing > src > pages > product > index.tsx > ProductPage > handleLogout
import Head from "next/head";
import { useRouter } from "next/router";
import { useEffect } from "react";
import useSWR from "swr";
import HeroSection from "../views/product/components/HeroSection";
import MainSection from "../views/product/components/MainSection";
import ShowProducts from "../views/product";

const fetcher = (url: string) => fetch(url).then((res) => res.json());

const ProductPage = () => {
  const { push } = useRouter();
  const { data, error, isLoading } = useSWR("/api/produk", fetcher);

  useEffect(() => {
    const loginStatus = localStorage.getItem("isLogin");
    if (loginStatus !== "true") {
      push("/auth/login");
    }
  }, [push]);

  const handleLogout = () => {
    localStorage.removeItem("isLogin");
    push("/auth/login");
  };

  if (isLoading) {
    return <p className="text-center text-gray-500 mt-10">Loading...</p>;
  }

  return (
    <>
      <Head>
        <title>Halaman Produk | My App</title>
      </Head>
      <div className="min-h-screen bg-gray-100">
        <HeroSection />
        <ShowProducts />
        <MainSection onLogout={handleLogout} />
      </div>
    </>
  );
};

export default ProductPage;
```

3. Buat folder swr pada utils dan tambahkan file dengan nama fetcher.js

```
P2 > Praktikum > next-routing > src > pages > utils > swr > fetcher.ts > default
const fetcher = (url: string) => fetch(url).then((res) => res.json());

export default fetcher;
```

4. Modifikasi file index.tsx pada folder pages/produk

```
import { useRouter } from "next/router";
import { useEffect } from "react";
import HeroSection from "../views/product/components/HeroSection";
import MainSection from "../views/product/components/MainSection";
import ShowProducts from "../views/product";
import useSWR from "swr";
import fetcher from "../utils/swr/fetcher";
import Head from "next/head";

// const fetcher = (url: string) => fetch(url).then((res) => res.json());

const ProductPage = () => {
  // const [isLogin, setIsLogin] = useState(false);
  const { push } = useRouter();

  const { data, error, isLoading } = useSWR("/api/product", fetcher);
  //cek apakah data, error, dan isLoading sudah benar

  useEffect(() => {
    const loginStatus = localStorage.getItem("isLogin");
    if (loginStatus !== "true") {
      push("/auth/login");
    }
  }, [push]);
```

5. Bandingkan:
   - **useEffect manual:** Logic kompleks (validation, transform), Non-fetch operations, Cleanup logic khusus
   - **SWR:** API calls sederhana, Data yang sering diminta, Perlu caching & revalidation

TUGAS PRAKTIKUM

1. Jelaskan perbedaan:
   - Client Side Rendering
     Pada CSR, halaman web dirender langsung di browser pengguna menggunakan JavaScript. Server hanya mengirimkan file dasar, lalu data diambil dan ditampilkan setelah halaman dimuat, sehingga interaksi terasa dinamis namun loading awal cenderung lebih lambat.

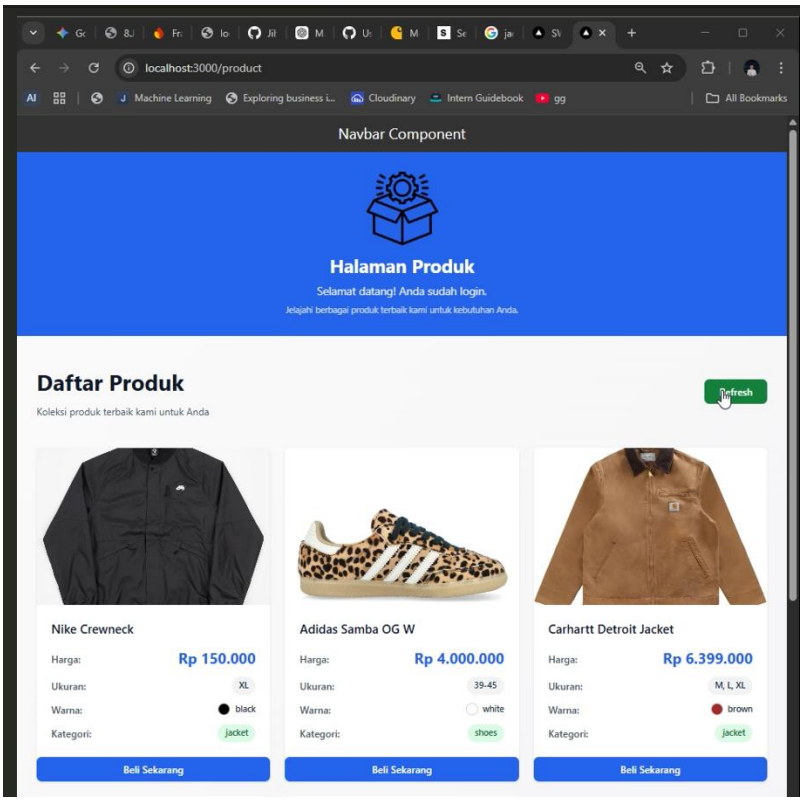
   - Server Side Rendering
     Pada SSR, halaman sudah dirender di server setiap kali ada permintaan dari pengguna. Hasilnya langsung dikirim dalam bentuk HTML lengkap, sehingga lebih cepat ditampilkan dan lebih baik untuk SEO, tetapi membebani server karena diproses berulang.

   - Static Site Generation

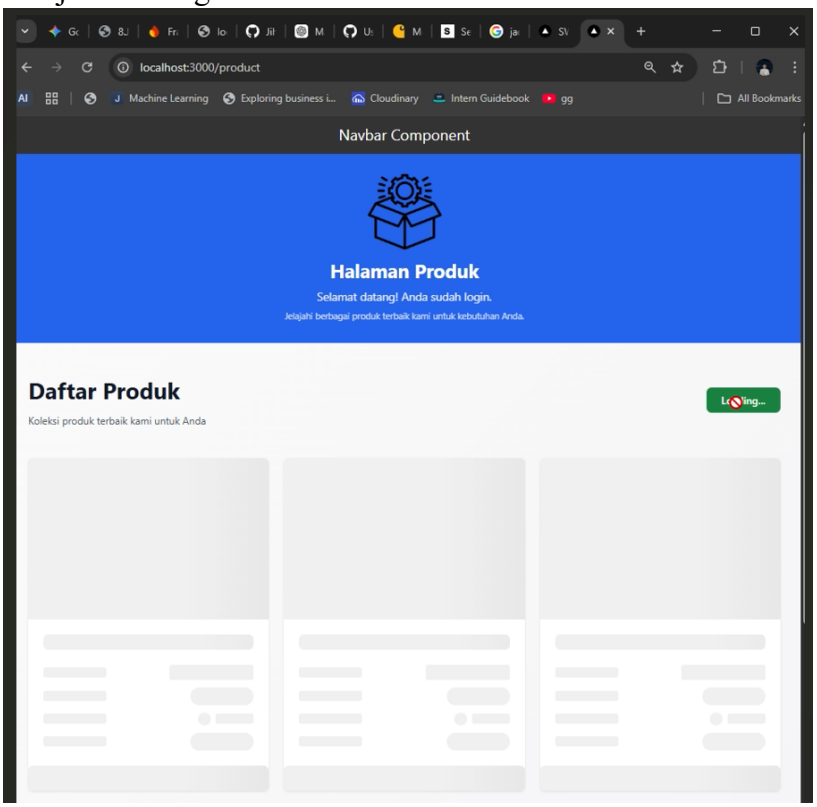
   - Pada SSG, halaman dibuat sebelumnya saat proses build dan disimpan sebagai file statis. Saat diakses, halaman langsung dikirim tanpa proses ulang, sehingga sangat cepat, namun kurang cocok untuk data yang sering berubah.

2. Buat halaman produk dengan:
   - Skeleton loading
   - Animasi

   UI awal



Setelah di refresh muncul skeleton loading dan animasi button yg berubah menjadi loading..



3. Refactor kode dari useEffect menjadi SWR.

```
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import HeroSection from "../views/product/components/HeroSection";
import MainSection from "../views/product/components/MainSection";
import ShowProducts from "../views/product";
import useSWR from "swr";
import fetcher from "../utils/swr/fetcher";
import Head from "next/head";

// const fetcher = (url: string) => fetch(url).then((res) => res.json());

const ProductPage = () => {
  // const [isLogin, setIsLogin] = useState(false);
  const { push } = useRouter();
  const [isRefreshing, setIsRefreshing] = useState(false);

  const { data, error, isLoading, isValidating, mutate } = useSWR( "/api/product", fetcher,);
  //cek apakah data, error, dan isLoading sudah benar
```